

Построение и исследование моделей прогнозирования спроса на аренду велосипедов на основе временных и метеорологических факторов

Лев Цыбань

Апрель 2026

Ключевые слова

регрессия, прогнозирование спроса, Bike Sharing Demand, градиентный бустинг, случайный лес, разведочный анализ данных, конструирование признаков

Содержание

1	Аннотация	4
2	Abstract	4
3	Введение	5
4	Постановка задачи и описание данных	6
5	Разведочный анализ данных	7
5.1	Анализ целевой переменной	7
5.2	Временные закономерности спроса	8
5.3	Погода и аномалии в признаках	11
5.4	Выводы	13
6	Базовая подготовка данных	13
7	Схема валидации	14
8	Базовые модели	15
9	Конструирование признаков	17
9.1	Формирование наборов признаков	17
9.2	Анализ избыточных признаков	17
9.3	Результаты экспериментов	18
10	Обработка категориальных признаков	18
11	Обработка аномалий в погодных данных	19
12	Эксперимент с отдельным прогнозированием	20

13 Подбор гиперпараметров	21
14 Анализ важности признаков и выводы	22
15 Заключение	24
Список литературы	25

1 Аннотация

В данной работе решается задача прогнозирования спроса на аренду велосипедов на основе датасета Bike Sharing Demand с платформы Kaggle. Цель проекта — минимизация ошибки RMSLE при предсказании почасового количества аренд. В ходе работы проведен разведочный анализ данных, выполнено конструирование признаков и логарифмирование целевой переменной. Для построения прогноза обучены и сравнены алгоритмы машинного обучения: решающее дерево, случайный лес и градиентный бустинг (XGBoost, LightGBM, CatBoost). Оценка качества моделей проводилась с помощью метода отложенной выборки, сохраняющей временную хронологию данных. В результате лучшие метрики показала модель CatBoost, что обусловлено ее внутренним алгоритмом обработки категориальных признаков.

2 Abstract

This paper addresses the problem of forecasting bike rental demand using the Bike Sharing Demand dataset from Kaggle. The goal of the project is to minimize the RMSLE metric when predicting hourly rentals. During the study, exploratory data analysis was performed, feature engineering was carried out, and the target variable was log-transformed. To build the forecast, various machine learning algorithms were trained and compared: decision tree, random forest, and gradient boosting (XGBoost, LightGBM, CatBoost). Model quality was evaluated using a time-based hold-out validation scheme. As a result, the CatBoost model demonstrated the best performance, which is explained by its native handling of categorical features.

3 Введение

Прогнозирование спроса на аренду велосипедов позволяет оптимизировать работу сервиса и более эффективно распределять инвентарь в течение суток. В данной работе рассматривается набор данных Bike Sharing Demand с платформы Kaggle. Задача заключается в предсказании почасового количества аренд на основе временных и погодных признаков. С точки зрения машинного обучения это классическая задача регрессии на табличных данных.

Основная трудность при работе с этим набором данных связана с нелинейным характером зависимостей. Спрос обладает выраженной цикличностью и меняется в зависимости от сочетания факторов. Например, влияние часа суток на целевую переменную напрямую зависит от того, является ли день рабочим или выходным. Кроме того, погодные условия могут значительно изменять привычные паттерны поведения пользователей.

По этой причине важным этапом работы является преобразование исходных данных и создание новых признаков, которые помогут моделям точнее улавливать эти закономерности. Также необходимо выбрать алгоритм, наиболее устойчивый к сложной структуре таких данных.

Целью работы является построение и сравнение нескольких моделей машинного обучения для получения наиболее точного прогноза спроса, а также оценка влияния различных методов подготовки признаков на итоговое качество предсказания.

4 Постановка задачи и описание данных

В рамках соревнования Bike Sharing Demand на платформе Kaggle предоставлены исторические данные об использовании системы городского велопроката Capital Bikeshare в Вашингтоне за 2011–2012 годы. Данные собраны с почасовой разбивкой.

Особенностью предоставленного набора данных является способ разделения на обучающую и тестовую выборки. Обучающая выборка содержит записи с 1-го по 19-е число каждого месяца, а тестовая — с 20-го числа до конца месяца. Цель задачи — спрогнозировать суммарное количество арендованных велосипедов за каждый час в тестовой выборке.

Каждая запись в наборе данных содержит следующие признаки:

- **datetime** — дата и время (с шагом в 1 час);
- **season** — время года (1 — весна, 2 — лето, 3 — осень, 4 — зима);
- **holiday** и **workingday** — бинарные флаги праздничного и рабочего дня;
- **weather** — оценка погодных условий от 1 (ясно) до 4 (сильный дождь/снег);
- **temp** и **atemp** — фактическая и ощущаемая температура (в градусах Цельсия);
- **humidity** — относительная влажность воздуха;
- **windspeed** — скорость ветра.

Кроме того, в обучающей выборке целевая переменная разбита на две составляющие:

- **casual** — количество аренд пользователями без регистрации;
- **registered** — количество аренд зарегистрированными пользователями;
- **count** — общее количество аренд (*casual* + *registered*), которое и требуется предсказывать.

Метрика оценки качества

Официальной метрикой соревнования является RMSLE (Root Mean Squared Logarithmic Error) — корень из среднеквадратичной логарифмической ошибки. Она вычисляется по следующей формуле:

$$\text{RMSLE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log(p_i + 1) - \log(a_i + 1))^2}$$

где:

- n — количество наблюдений в тестовой выборке;
- p_i — предсказанное моделью количество аренд;
- a_i — фактическое количество аренд;
- $\log(x)$ — натуральный логарифм.

Использование логарифмической ошибки вместо обычной MSE (Mean Squared Error) обусловлено тем, что RMSLE штрафует за относительную ошибку прогноза, а не за абсолютную. Добавление единицы под знаком логарифма необходимо для того, чтобы избежать вычисления логарифма от нуля в те часы, когда велосипеды никто не арендовал.

5 Разведочный анализ данных

В этом разделе нужно понять, от чего в данных сильнее всего зависит спрос. Для этого посмотрим на распределение целевой переменной, на временные закономерности и на несколько важных особенностей признаков.

5.1 Анализ целевой переменной

Сначала посмотрим на целевую переменную `count`, то есть на общее количество аренд за час.

В соревновании Bike Sharing Demand ответ оценивается по метрике RMSLE. Это значит, что важны не сами значения спроса, а их логарифмы. Поэтому

предсказывать `count` напрямую не совсем правильно. Гораздо логичнее сразу обучать модель на величине

$$y = \log(\text{count} + 1).$$

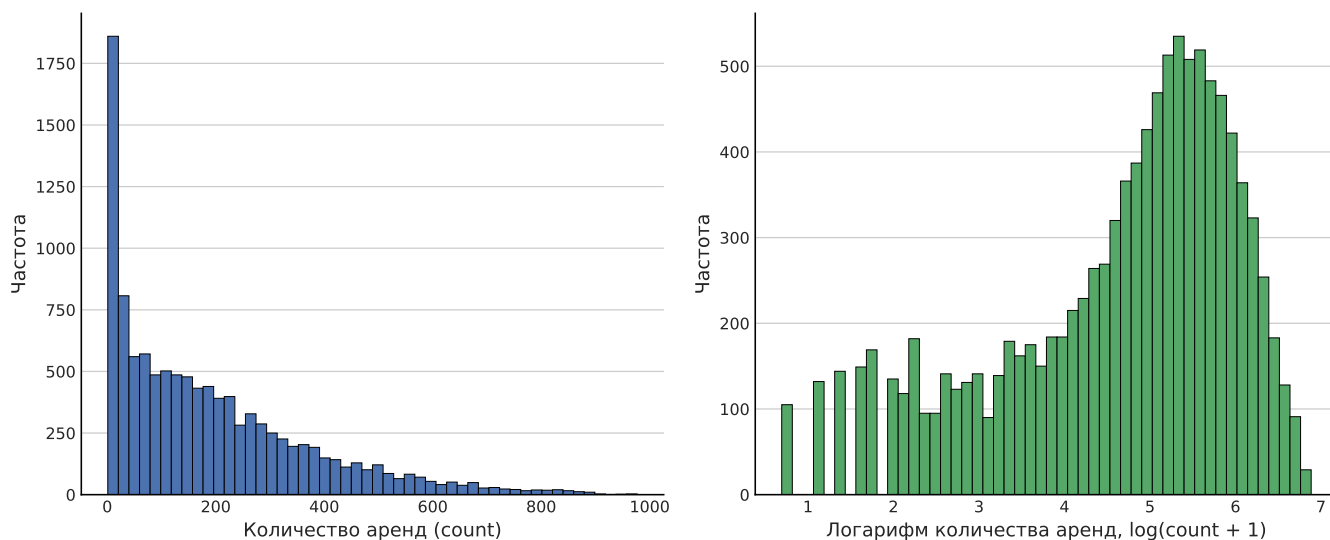


Рис. 5.1.1: Распределение целевой переменной: исходное значение (слева) и логарифмированное $\log(\text{count} + 1)$ (справа).

По рисунку 5.1.1 видно, что исходное распределение `count` несимметрично: небольшие значения встречаются часто, а большие — редко. После логарифмирования оно становится более симметричным. Это ещё одна причина, почему такой вариант удобен.

Дальше будем обучать модели именно на $\log(\text{count} + 1)$, а затем переводить предсказания обратно в обычный масштаб.

5.2 Временные закономерности спроса

Главные закономерности в этом датасете связаны со временем. Спрос сильно меняется в зависимости от часа, типа дня и времени года.

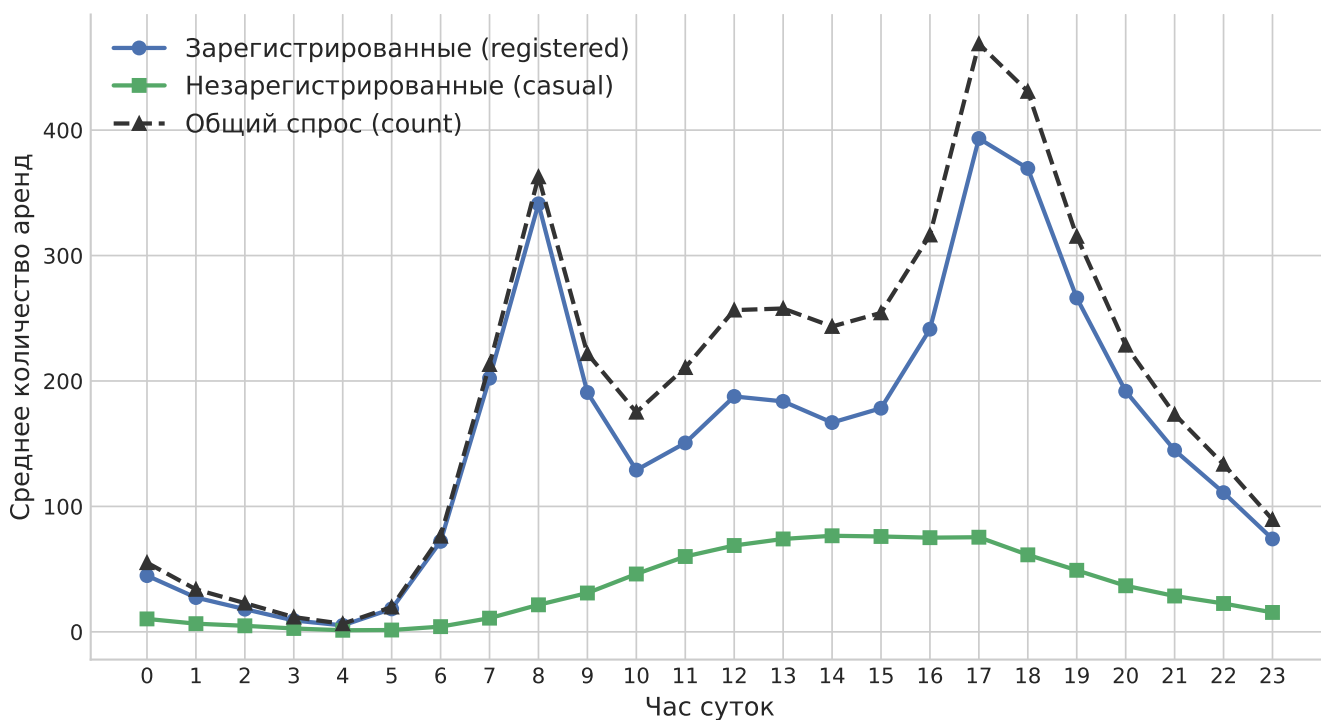


Рис. 5.2.1: Среднее количество аренд по часам суток: общее число аренд (*count*), зарегистрированные (*registered*) и незарегистрированные (*casual*) пользователи.

На рисунке 5.2.1 видно, что общий спрос складывается из двух разных типов поведения. Зарегистрированные пользователи чаще арендуют велосипеды в часы пик — утром и вечером. Незарегистрированные пользователи ведут себя иначе: их спрос растёт более плавно и достигает максимума днём.

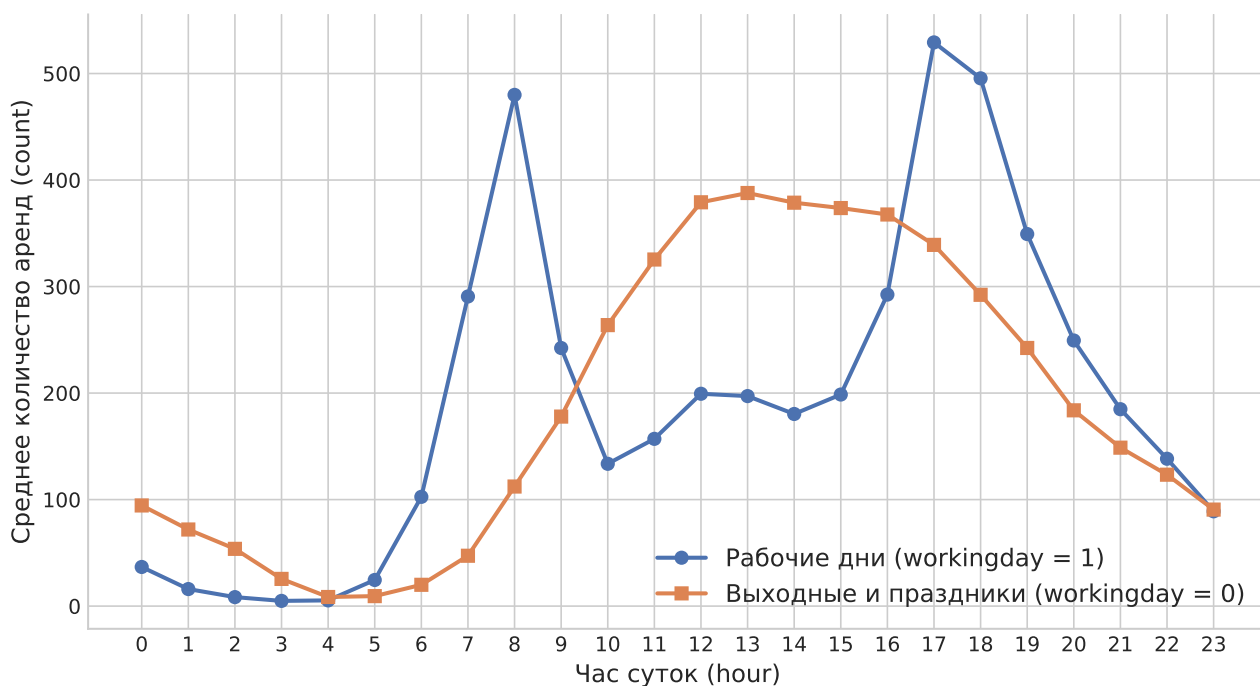


Рис. 5.2.2: Среднее количество аренд по часам в рабочие и выходные дни.

На рисунке 5.2.2 видно, что рабочие и выходные дни отличаются очень сильно. В рабочие дни есть два выраженных пика: утром и вечером. В выходные спрос растёт более плавно и максимален в середине дня. Это означает, что один и тот же час может по-разному влиять на спрос в зависимости от типа дня.

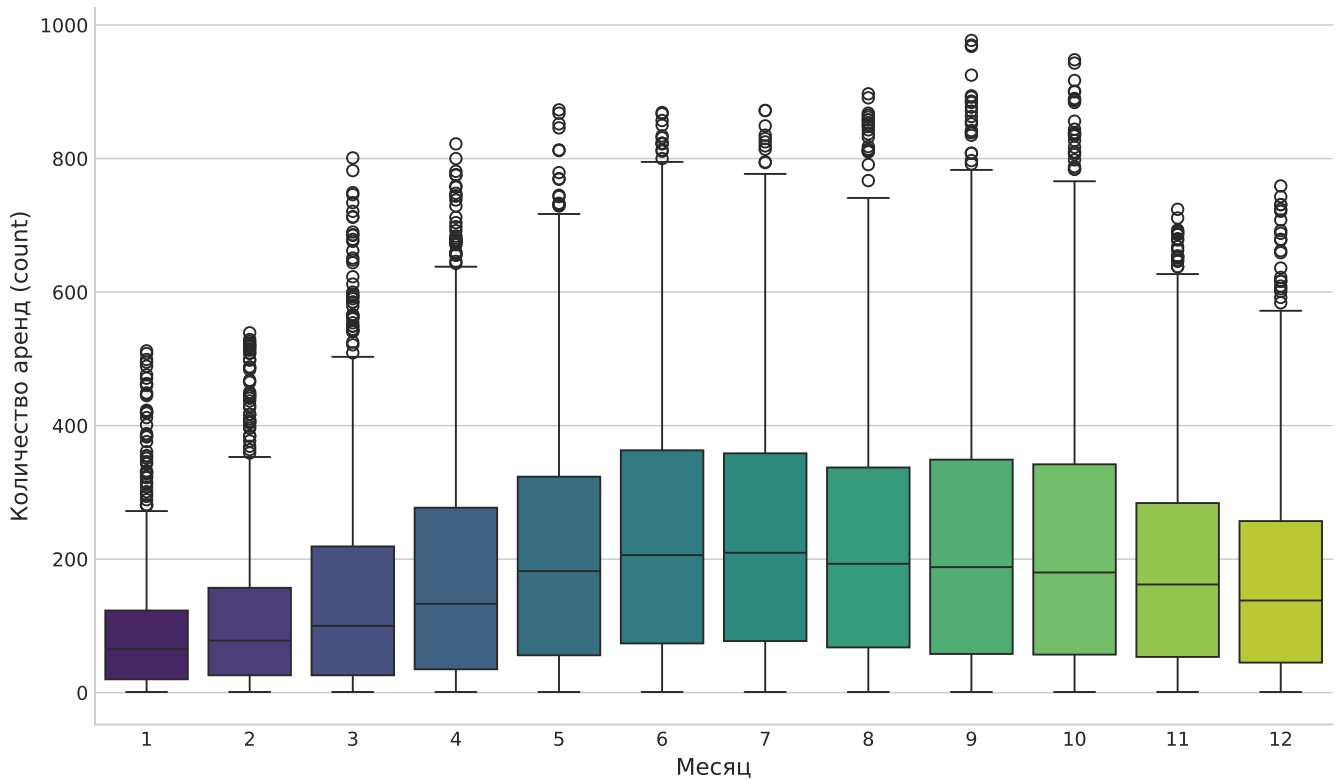


Рис. 5.2.3: Распределение спроса по месяцам.

На рисунке 5.2.3 заметна сезонность: от весны к лету спрос увеличивается, а зимой снова снижается.

На рисунке 5.2.4 видно, что в 2012 году средний спрос в большинстве месяцев выше, чем в 2011 году. Это указывает на общий рост популярности сервиса, который также полезно учитывать в модели.



Рис. 5.2.4: Среднее количество аренд по месяцам отдельно для 2011 и 2012 годов.

5.3 Погода и аномалии в признаках

Погода, как и ожидалось, заметно влияет на спрос. В ясные тёплые дни люди берут велосипеды чаще, а при дожде или снеге прокаты почти исчезают. Но при более детальном изучении погодных данных выявились особенности, требующие внимания на этапе подготовки признаков.

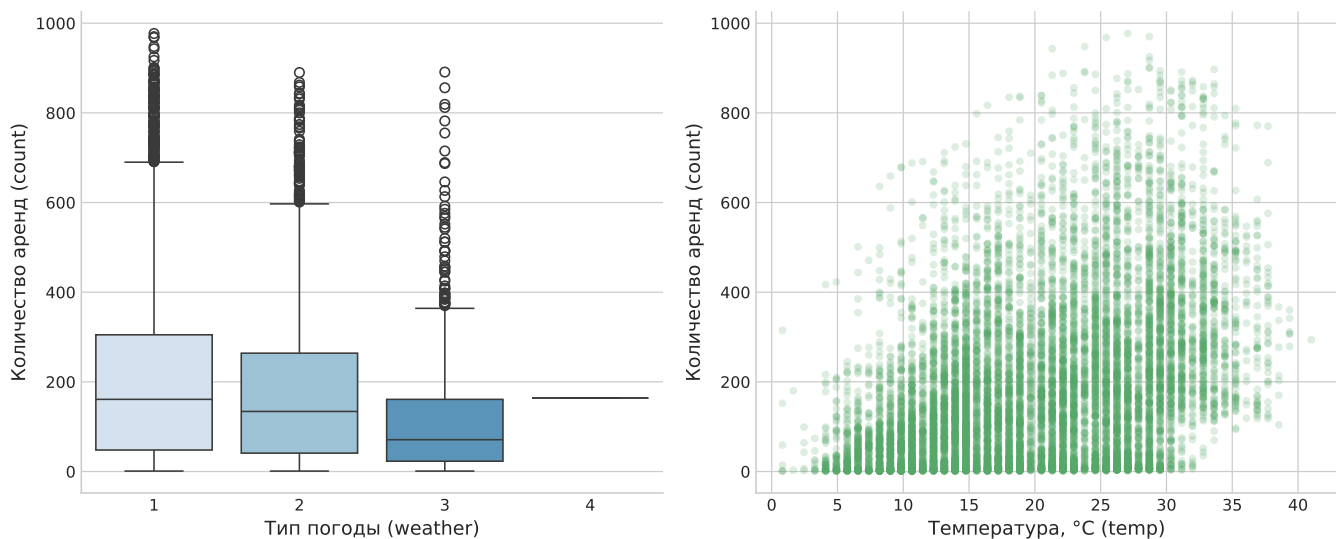


Рис. 5.3.1: Слева: среднее количество аренд в зависимости от категории погоды. Справа: зависимость спроса от температуры.

На рисунке 5.3.1 видно, что при ясной погоде (`weather = 1`) спрос в среднем выше всего, а при ухудшении условий он предсказуемо падает. Также

хорошо заметно, что с ростом температуры число аренд в целом увеличивается — люди охотнее катаются в тёплую погоду.

При анализе температурных признаков `temp` и `atemp` (ощущаемая температура) обнаруживается их очень высокая корреляция. Для классических линейных моделей такое дублирование создало бы проблему мультиколлинеарности. Хотя алгоритмы на основе деревьев решений устойчивы к подобным эффектам, подавать в модель два почти идентичных признака может быть неоптимально. На этапе конструирования признаков будет целесообразно проверить использование их разности (`temp - atemp`), которая физически отражает уровень погодного дискомфорта (например, из-за влажности или ветра).

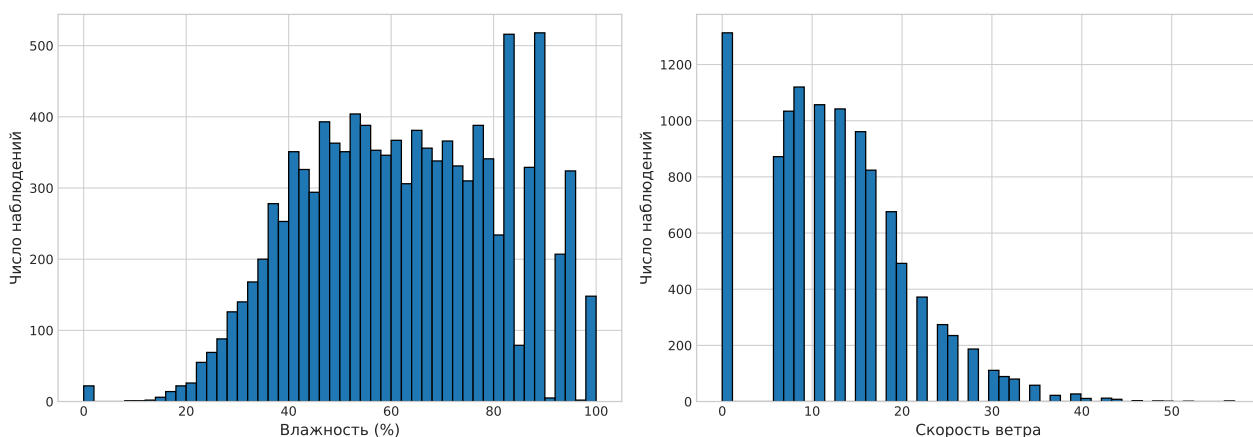


Рис. 5.3.2: Распределения признаков влажности (`humidity`) и скорости ветра (`windspeed`).

Важную аномалию можно заметить в признаках `windspeed` и `humidity`. На рисунке 5.3.2 видно, что у распределения скорости ветра присутствует неестественно большой пик ровно в нуле. Абсолютный штиль встречается крайне редко. Вероятнее всего, нулями были заполнены пропуски в данных, возникшие из-за сбоев метеорологических датчиков. У влажности также присутствует пик в нуле, но нулевых значений мало, так что на точность предсказаний это повлиять не должно.

Наличие таких скрытых пропусков требует осторожного подхода. С одной стороны, алгоритм может найти ложную закономерность. С другой стороны, сами поломки датчиков часто происходят при экстремальных погодных условиях (например, обледенение), и в таком случае нули могут служить полезным скрытым сигналом о плохой погоде. В ходе экспериментов планируется проверить обе гипотезы: как обучение на исходных данных, так и

попытку восстановить реальную скорость ветра с помощью вспомогательной ML-модели.

5.4 Выводы

Подводя итог разведочному анализу, выделим главные особенности, которые нужно учесть при подготовке признаков и обучении моделей:

- Для корректной оптимизации турнирной метрики RMSLE целевая переменная `count` будет логарифмирована перед обучением алгоритмов.
- Анализ показал сильную зависимость спроса от календаря. На основе метки времени (`datetime`) потребуется выделить отдельные признаки: час, день недели, месяц и год.
- Поскольку суточные графики спроса в будни и выходные кардинально отличаются, целесообразно проверить признаки взаимодействия между часом суток и типом дня.
- Паттерны поведения `casual` и `registered` пользователей принципиально разные. В ходе экспериментов планируется проверить раздельное прогнозирование компонент `casual` и `registered`
- В данных о погоде есть дублирующие друг друга метрики (`temp` и `atemp`) и аномальные нули в `humidity` и `windspeed`. На этапе подготовки данных планируется обработка этих признаков.

6 Базовая подготовка данных

Перед тем как обучать алгоритмы, исходные данные нужно подготовить: убрать лишнее, извлечь полезную информацию из дат и преобразовать целевую переменную. На этом этапе мы сформируем базовый набор признаков, с которым будем сравнивать все дальнейшие улучшения.

В первую очередь из обучающей выборки были удалены столбцы `casual` и `registered`. Общий спрос (`count`) является их суммой, поэтому оставлять их в качестве признаков нельзя — это приведёт к прямой утечке ответов.

Согласно правилам соревнования и выводам из разведочного анализа, целевая переменная `count` была преобразована с помощью натурального логарифма: $y = \log(\text{count} + 1)$. В экспериментах, где спрос предсказывался отдельно, логарифмировались величины `casual` и `registered`. Такое преобразование позволяет стандартным алгоритмам минимизировать обычную среднеквадратичную ошибку (MSE), что математически эквивалентно оптимизации турнирной метрики RMSLE.

Единственным нечисловым столбцом в данных была метка времени `datetime`. Алгоритмы на основе деревьев решений не умеют работать со строками, поэтому столбец был переведён в формат даты. Из него мы извлекли четыре базовых календарных признака: `hour` (час суток), `weekday` (день недели), `month` (месяц) и `year` (год). После этого исходный столбец `datetime` был удалён как из обучающей, так и из тестовой выборки.

Отдельно стоит сказать про день месяца (`day`). В базовой модели он не использовался. Это связано с тем, как организаторы разделили данные: обучающая выборка содержит только дни с 1-го по 19-е число, а тестовая — с 20-го числа до конца месяца. Если оставить признак `day`, модель научится предсказывать только первую половину месяца, а на тестовых данных (дни 20-31) будет ошибаться.

В результате была сформирована отправная точка для экспериментов — минимальный набор данных, включающий исходную погоду, флаги праздников и четыре базовых признака времени.

7 Схема валидации

Выбор правильного способа оценки качества модели (схемы валидации) — один из самых важных этапов, особенно при работе с временными рядами. Обычное случайное перемешивание данных (K-Fold) здесь не подходит.

Если разбить данные случайно, в обучающую выборку могут попасть данные за 14:00 и 16:00 одного дня, а в проверочную — за 15:00. Модель просто усреднит значения соседних часов и покажет отличный результат, но на реальном тесте, где нужно предсказывать на неделю вперед, она провалится.

Стандартное разбиение по времени (`TimeSeriesSplit`) в этой задаче тоже работает плохо. Особенность датасета в том, что обучающая выборка содер-

жит данные с 1-го по 19-е число каждого месяца, а тестовая — с 20-го числа до конца месяца. То есть задача состоит не в предсказании далекого будущего, а в регулярном прогнозировании конца каждого месяца на основе его начала. При использовании `TimeSeriesSplit` на первых этапах проверки модель не увидела бы данные за все сезоны и годы, хотя в реальности они ей доступны.

Поэтому была реализована схема валидации с отложенной выборкой, полностью повторяющая логику соревнования. Из обучающей выборки (дни 1–19) в качестве проверочного набора были выделены последние доступные дни — с 16-го по 19-е число каждого месяца. Дни с 1-го по 15-е число использовались для обучения.

Такой подход позволил получить честную и стабильную оценку качества моделей, которая хорошо соотносится с результатами на платформе Kaggle.

8 Базовые модели

Для объективной оценки результатов необходимо задать отправные точки. Прогноз медианным значением дает ошибку $\text{RMSLE} \approx 1.4$. Лучшие решения в рамках данного соревнования достигают метрики ≈ 0.33 . Это соответствует ошибке приблизительно в 1.4 раза

В качестве базовых моделей были протестированы пять алгоритмов: решающее дерево (Decision Tree), случайный лес (Random Forest) и три реализации градиентного бустинга (LightGBM, CatBoost, XGBoost). Обучение проводилось на минимальном наборе признаков: исходные погодные данные и базовые компоненты времени (`hour`, `weekday`, `month`, `year`). Автоматический подбор гиперпараметров на данном этапе не проводился; использовались стандартные значения, ограничивающие переобучение (например, неглубокие деревья и `learning_rate` ≈ 0.03).

Результаты локальной валидации и оценки на публичном лидерборде Kaggle представлены на рисунке 8.0.1.

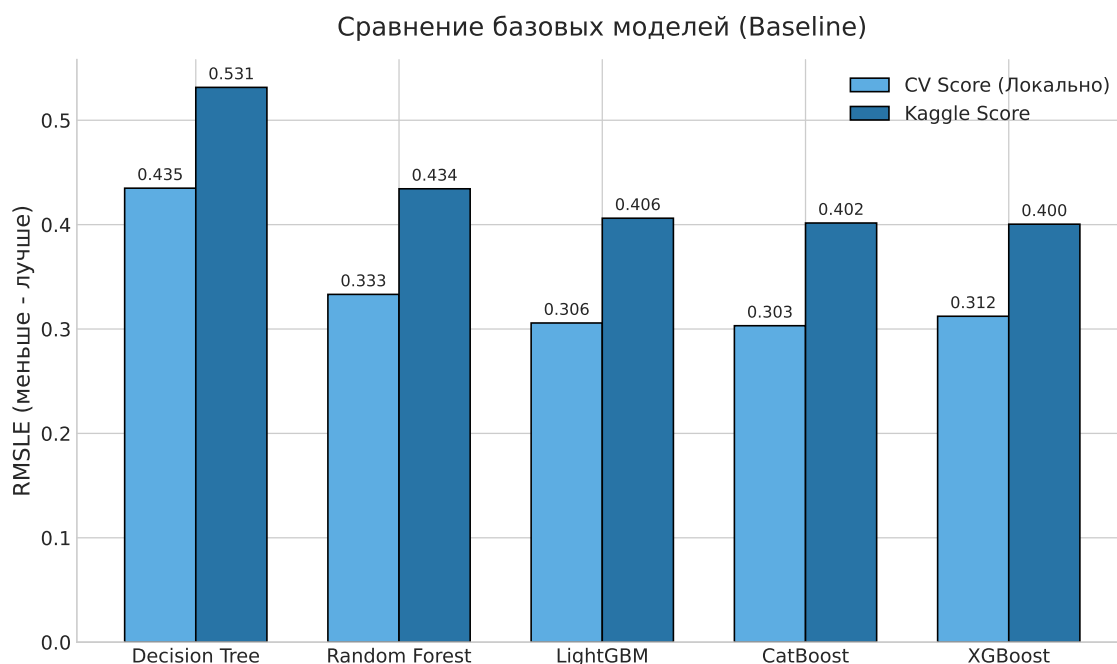


Рис. 8.0.1: Сравнение базовых моделей: ошибка на локальной валидации и на платформе Kaggle.

Из полученных результатов следуют три основных вывода:

1. Одиночное решающее дерево предсказуемо показало наихудший результат из-за склонности к переобучению и неспособности улавливать сложные зависимости без ансамблирования.
2. Алгоритмы градиентного бустинга продемонстрировали схожую эффективность, достигнув отметки ≈ 0.40 на тестовой выборке. Лучшим алгоритмом оказался XGBoost (0.40043).
3. Стабильный разрыв между локальной ошибкой ($CV \approx 0.31$) и результатом на Kaggle (≈ 0.40) объясняется увеличенным горизонтом прогнозирования: 11–12 дней для тестовой выборки против 4 дней на валидации.

Результат XGBoost (0.40043) зафиксирован как базовая метрика. Это соответствует ошибке в среднем в приблизительно 1.5 раза. Эффективность дальнейшего конструирования признаков будет оцениваться относительно этого значения.

9 Конструирование признаков

Конструирование признаков проводилось поэтапно на основе выводов разведочного анализа. Были сформированы два основных набора (FE v1 и FE v2).

9.1 Формирование наборов признаков

Набор FE v1. Включает бинарные индикаторы суточной активности: `is_weekend`, `is_night`, `morning_rush` (07:00–09:00) и `evening_rush` (17:00–19:00), а также признак взаимодействия `hour_workingday`. Цель набора — указание модели на важные части дня, а также разделение рабочих и нерабочих дней.

Набор FE v2. Расширяет FE v1 циклическим кодированием времени (синусы и косинусы для `hour`, `weekday`, `month`) для устранения разрывов на границах периодов (например, между 23:00 и 00:00). Дополнительно введены производные погодные метрики: разность температур (`temp_atemp_diff`) и индекс дискомфорта (`discomfort_index = temp \times humidity/100`).

9.2 Анализ избыточных признаков

Был протестирован и третий, расширенный набор, включающий попарные произведения погодных факторов и линейный тренд времени (`days_from_start`).

Эксперимент показал резкое ухудшение метрики на платформе Kaggle до ≈ 0.42 при неизменной ошибке на локальной валидации (≈ 0.31). Это объясняется двумя факторами:

1. **Отсутствие экстраполяции у деревьев решений.** Признак `days_from_start` монотонно возрастает. На тестовой выборке появляются значения, которых не было в обучении, из-за чего деревья не могут выдать адекватный прогноз.
2. **Переобучение.** Избыточное расширение признакового пространства привело к тому, что алгоритмы начали находить ложные закономерности в локальном шуме обучающей выборки.

Таким образом, для градиентного бустинга компактный и физически обоснованный набор признаков оказался эффективнее избыточного.

9.3 Результаты экспериментов

Результаты тестирования базовых моделей на наборах FE v1 и FE v2 представлены в таблице 9.3.1.

Модель	Набор признаков	RMSLE (Валидация)	RMSLE (Kaggle)
XGBoost	base	0.31222	0.40043
XGBoost	FE v1	0.31047	0.39935
XGBoost	FE v2	0.31326	0.39918
CatBoost	base	0.30316	0.40156
CatBoost	FE v1	0.30985	0.39314
CatBoost	FE v2	0.30702	0.38814
LightGBM	base	0.30578	0.40615
LightGBM	FE v1	0.31352	0.39849
LightGBM	FE v2	0.30949	0.40040

Таблица 9.3.1: Сравнение моделей на различных наборах признаков.

Наибольший прирост качества от конструирования признаков показала модель CatBoost на наборе FE v2, достигнув RMSLE 0.38814 на публичном лидерборде. Для XGBoost и LightGBM новые признаки не дали существенного улучшения. Набор FE v2 зафиксирован как оптимальный для итоговой настройки модели.

10 Обработка категориальных признаков

В исходном формате категориальные признаки (`weather`, `season`, `month` и т. д.) кодировались целыми числами, и модели воспринимали их как непрерывные величины. Вместо этого каждая алгоритмическая реализация была проверена с явным указанием категорий через встроенные механизмы библиотек.

Для CatBoost использовался параметр `cat_features`, для LightGBM — перевод колонок в тип `category`, для XGBoost — параметр `enable_categorical`

в сочетании с `tree_method='hist'`.

Результаты представлены в таблице 10.0.1.

Модель	RMSLE (Валидация)	RMSLE (Kaggle)
CatBoost (category)	0.30839	0.37982
XGBoost (category)	0.31350	0.41396
LightGBM (category)	0.30977	0.40600

Таблица 10.0.1: Результаты моделей с явным указанием категориальных признаков.

Наибольший прирост показал CatBoost: ошибка на Kaggle снизилась с 0.38814 до 0.37982. Для XGBoost и LightGBM использование категорий не дало положительного эффекта, а в случае XGBoost даже ухудшило результат. Это объясняется тем, что алгоритм целевого кодирования (Target Encoding), используемый внутри CatBoost, лучше подходит для данного типа данных, чем подходы на основе сортировки категорий, применяемые в других реализациях.

11 Обработка аномалий в погодных данных

На этапе разведочного анализа были выявлены физически невозможные нулевые значения в признаках относительной влажности (`humidity`) и скорости ветра (`windspeed`). Детальный анализ показал, что нулевая влажность зафиксирована лишь в течение одних суток за весь период наблюдений, что является очевидным сбоем датчика. Ввиду малой доли таких наблюдений алгоритмическое восстановление данного признака нецелесообразно.

Напротив, нулевые значения `windspeed` составляют значительную часть выборки. Была выдвинута гипотеза, что алгоритмическое восстановление реальной скорости ветра для этих записей снизит зашумленность данных и улучшит качество прогноза.

Для заполнения нулей была применена вспомогательная модель Random Forest, обученная на корректных записях (`windspeed` $\neq 0$) с использованием остальных временных и погодных признаков. Обновленный датасет был протестирован на лучших конфигурациях моделей (FE v2 с явным указанием категориальных признаков).

Результаты эксперимента представлены в таблице 11.0.1.

Модель	RMSLE (Валидация)	RMSLE (Kaggle)
CatBoost (wind fixed)	0.30814	0.38672
XGBoost (wind fixed)	0.31265	0.40417
LightGBM (wind fixed)	0.31303	0.40556

Таблица 11.0.1: Результаты моделей после алгоритмического восстановления скорости ветра.

Эксперимент показал согласованное ухудшение метрики на платформе Kaggle для всех алгоритмов (метрика CatBoost ухудшилась с 0.37982 до 0.38672).

Подобная деградация качества объясняется тем, что для деревьев решений само наличие нуля выступает сильным скрытым сигналом. Зачастую датчики выходят из строя при экстремальных погодных условиях (например, обледенение), когда спрос на велосипеды объективно минимален. Искусственное заполнение этих пропусков синтетическими «нормальными» значениями уничтожило этот паттерн, заставив модели завышать прогноз в периоды непогоды. В итоговом пайплайне погодные признаки были оставлены без изменений.

12 Эксперимент с раздельным прогнозированием

На этапе разведочного анализа была выдвинута гипотеза о целесообразности раздельного прогнозирования спроса для зарегистрированных (**registered**) и незарегистрированных (**casual**) пользователей с последующим суммированием результатов.

Для проверки этой идеи была использована лучшая конфигурация: алгоритм CatBoost на наборе признаков FE v2 с явным указанием категориальных переменных. Предполагалось, что обучение двух независимых моделей позволит точнее учесть разную природу спроса: стабильные рабочие поездки и волатильный прогулочный трафик.

Результаты эксперимента на тестовой выборке Kaggle показали ошибку 0.38556, что оказалось хуже показателя одиночной модели (0.37982). Такое

снижение точности подтверждает эффект накопления ошибки: при сложении двух прогнозов их независимые погрешности суммируются, что в данном случае нивелировало преимущество раздельного подхода. На основе полученных данных было принято решение вернуться к прямому прогнозированию общей переменной `count`.

13 Подбор гиперпараметров

На финальном этапе была предпринята попытка улучшить метрику лучшей модели (CatBoost на наборе FE v2) за счет подбора гиперпараметров.

Сначала использовалась библиотека автоматической оптимизации `Optuna`. Поиск параметров проводился по локальной отложенной выборке (дни 16–19). Алгоритм нашел настройки, которые сильно уменьшили локальную ошибку ($\text{RMSLE} \approx 0.302$). Однако на тестовой выборке Kaggle результат, наоборот, ухудшился до ≈ 0.395 .

Причина такого поведения — переобучение. `Optuna` подобрала параметры (например, глубокие деревья), которые идеально запомнили особенности середины месяца, но модель потеряла способность обобщать данные на конец месяца (дни 20–31), где условия могли слегка измениться.

Чтобы решить эту проблему, был проведен ручной перебор параметров. Модель намеренно упрощалась: уменьшалась глубина деревьев (`depth` с 6 до 5), снижалась скорость обучения (`learning_rate`) и усиливалась регуляризация. Ожидалось, что это защитит модель от переобучения и сделает прогноз более стабильным. Тем не менее, ручной подбор также не смог превзойти предыдущие результаты.

В итоге эксперименты показали, что изначальные, стандартные параметры средней сложности (`iterations=800`, `learning_rate=0.03`, `depth=6`) являются наиболее сбалансированными и устойчивыми для данной задачи. Они не дают модели переобучиться, но позволяют находить нужные закономерности.

14 Анализ важности признаков и выводы

Для интерпретации логики итоговой модели (CatBoost на наборе FE v2) была построена диаграмма важности признаков. Алгоритм оценивал вклад каждой переменной в общее снижение ошибки прогноза. Результаты для десяти наиболее значимых признаков представлены на рисунке 14.0.1.

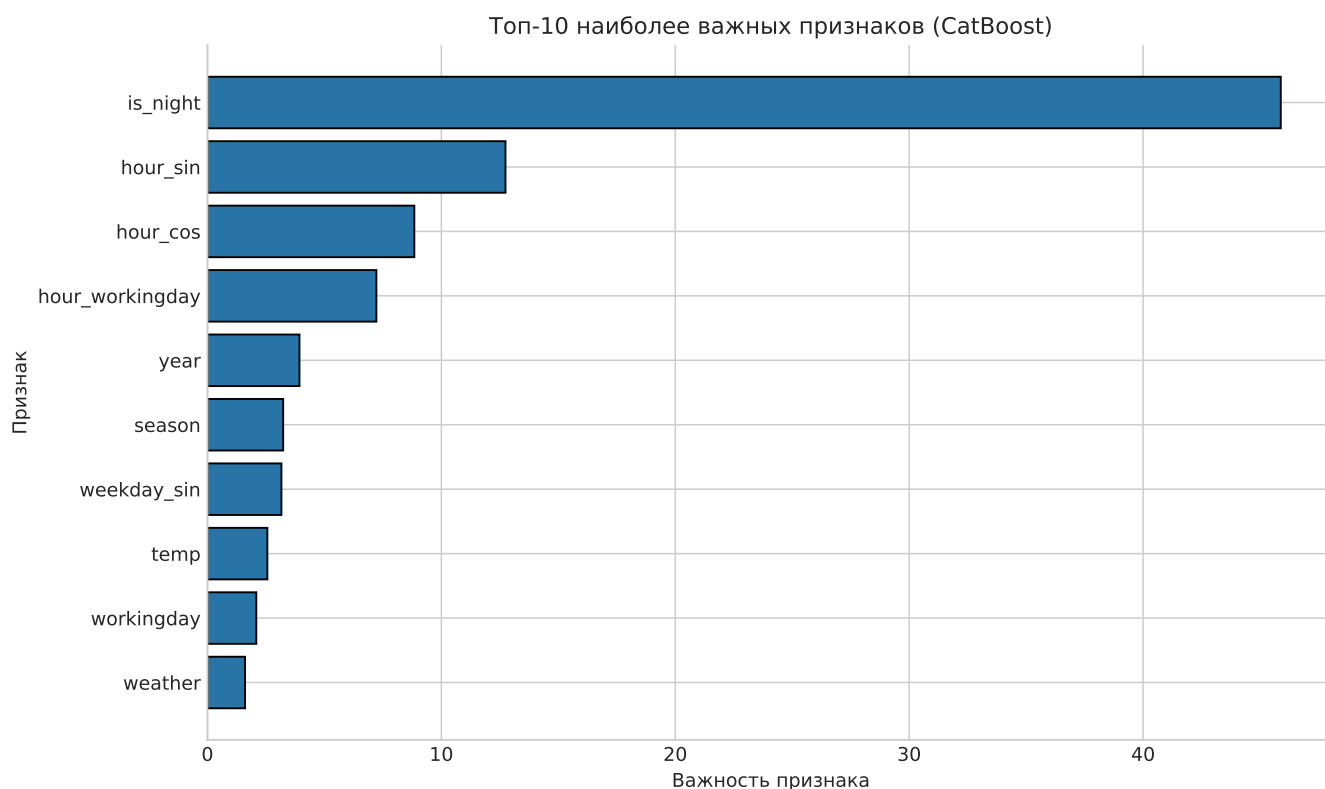


Рис. 14.0.1: Топ-10 наиболее важных признаков в итоговой модели CatBoost.

Как видно из графика, абсолютно доминирующим фактором стал бинарный признак `is_night` (около 46% важности). Модель использует его как первичный фильтр, поскольку в ночные часы спрос падает практически до нуля независимо от погоды и дня недели.

Значительную долю важности заняли циклические признаки времени (`hour_sin`, `hour_cos` и `weekday_sin`). Это доказывает высокую эффективность циклического кодирования: представление времени в виде непрерывного цикла позволило деревьям решений лучше улавливать суточные и недельные паттерны.

Присутствие в топе признака `hour_workingday` (более 7%) подтверждает ключевую гипотезу разведочного анализа: профиль спроса кардинально меняется в зависимости от того, является ли день рабочим, и модель активно использует это взаимодействие. Признак `year` успешно выступил в роли

категориального маркера глобального растущего тренда популярности сервиса. Погодные метрики (**temp, weather**) оказались в конце десятки: они выступают в роли корректирующих факторов, уточняя прогноз после того, как сформирована основная календарная база.

Общие выводы по этапу моделирования:

- Алгоритмы градиентного бустинга (в первую очередь CatBoost) показали превосходство над классическими деревьями решений в задаче прогнозирования нелинейного спроса.
- Наибольший прирост качества обеспечило не избыточное увеличение числа признаков, а использование циклических признаков времени (синусов и косинусов) и корректная обработка категориальных переменных внутренними механизмами модели.
- Ограничение сложности алгоритма (глубина деревьев, доля подвыборки) оказалось критически важным для предотвращения переобучения и успешной экстраполяции прогноза на конец месяца.

Итоговая модель достигла результата **RMSLE = 0.37982** на тестовой выборке платформы Kaggle, что соответствует **108-му месту** среди более чем 3200 команд исторического лидерборда (попадание в **топ-3.5%** лучших решений).

15 Заключение

В данной работе была рассмотрена задача прогнозирования почасового спроса на аренду велосипедов на основе временных и погодных признаков. Были проведены разведочный анализ данных, базовая подготовка признаков, сравнение нескольких моделей машинного обучения и серия экспериментов по конструированию новых признаков.

В ходе работы было показано, что наилучшие результаты в данной задаче дают методы градиентного бустинга, а наиболее эффективной моделью оказался CatBoost. Существенное влияние на качество прогноза оказали корректная обработка категориальных признаков и использование дополнительных временных признаков. При этом избыточное усложнение признакового пространства, а также агрессивный подбор гиперпараметров приводили к ухудшению результата из-за переобучения.

Итоговая модель достигла значения RMSLE 0.37982 на платформе Kaggle, что соответствует 108-му месту в историческом лидерборде соревнования. Полученные результаты показывают, что даже при сравнительно простом наборе данных можно добиться высокого качества прогноза за счёт последовательного анализа, аккуратного подбора признаков и правильного выбора алгоритма.

Список литературы

- [1] Bike sharing demand.
URL: <https://www.kaggle.com/competitions/bike-sharing-demand>.
- [2] Catboost documentation.
URL: <https://catboost.ai/docs>.
- [3] Lightgbm documentation.
URL: <https://lightgbm.readthedocs.io/en/latest>.
- [4] Optuna: A hyperparameter optimization framework.
URL: <https://optuna.readthedocs.io/en/stable>.
- [5] scikit-learn: machine learning in python.
URL: <https://scikit-learn.org/stable>.
- [6] Xgboost documentation.
URL: <https://xgboost.readthedocs.io/en/stable>.
- [7] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system.
In *Proceedings of the 22nd ACM SIGKDD International Conference*, 2016.
URL: <https://arxiv.org/abs/1603.02754>.
- [8] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. Catboost: unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, 2018.
URL: <https://arxiv.org/abs/1706.09516>.